

COSC3506/ITEC3506

Software Engineering

Module 5 Lesson 1

Domain Modelling

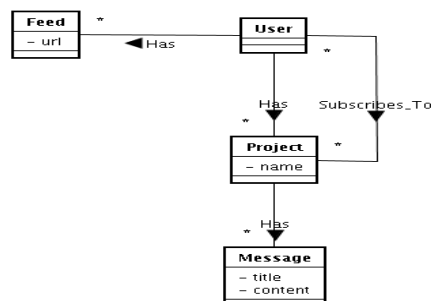
Dr. Simon Xu

Copyright © 2020 by Dr. Simon Xu,
CC BY-SA 4.0

1

Domain Model (conceptual model)

- A domain model is a *visual representation* of conceptual classes or real-situation objects in a domain.
- A domain model is illustrated with a *class diagram* in which *no operations (method signatures) are defined*.



Copyright © 2020 by Dr. Simon Xu,
CC BY-SA 4.0

2

Domain Model

- **Domain Model is a visualization of things in a real-situation domain of interest, *not* of software objects such as Java or C# classes, or software objects with responsibilities.**
- **Are those domain concepts?**
 - Software artifacts, such as a **window** or a **database**,
 - Responsibilities or methods.

Copyright © 2020 by Dr. Simon Xu,

3

Domain Model

Figure 9.3. A domain model shows real-situation conceptual classes, not software classes.

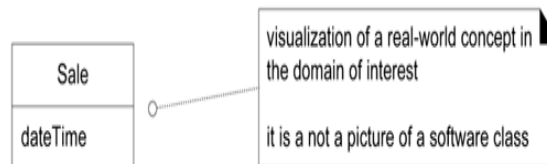
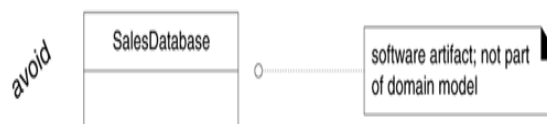


Figure 9.4. A domain model does not show software artifacts or classes.



Copyright © 2020 by Dr. Simon Xu,

4

Domain Model

- Informally, a **conceptual class** is an idea, thing, or object.

Copyright © 2020 by Dr. Simon Xu,
CC BY-SA

5

Domain Concept

- **Examples of domain concept/entities:**

– **Person:**

– **Place:**

– **Object:**

– **Event:**

– **Concept:**



Copyright © 2020 by Dr. Simon Xu,
CC BY-SA

6

Domain Concept

- **Guidelines for naming and defining class/entity types:**
 - An concept class/entity type name is a singular **noun**
 - An entity name should be **concise**
 - Event entity types should be named for the **result** of the event, not the activity or process of the event.

Copyright © 2020 by Dr. Simon Xu,

7

Domain Model

- **Domain model To help understanding and more...**
 - To support a lower gap between the software representation and our mental model of the domain.

Copyright © 2020 by Dr. Simon Xu,

8

Domain Model

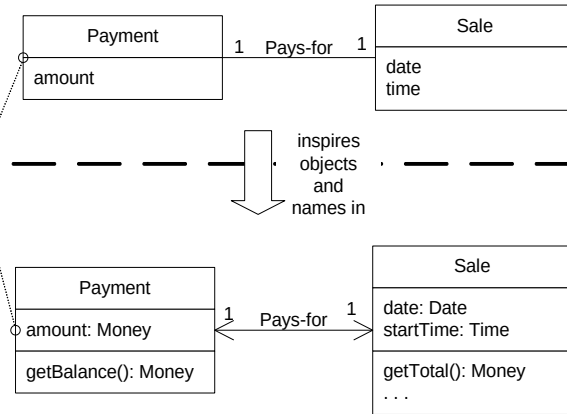
supports a low representational gap between our mental and software models.

Stakeholder's view of the noteworthy concepts in the domain.

A Payment in the Domain Model is a concept, but a Payment in the Design Model is a software class. They are not the same thing, but the former *inspired* the naming and definition of the latter.

This reduces the representational gap.

This is one of the big ideas in object technology.



UP Design Model

The object-oriented developer has taken inspiration from the real world domain in creating software classes.

here's a source-code payroll program written in 1953:
10000101010001111010101010100010101010101111010101

Copyright © 2020 by Dr. Simon Xu, CS144

9

Domain Model

• How to create a domain model?

Four steps

- Step 1
- Step 2
- Step 3
- Step 4

Copyright © 2020 by Dr. Simon Xu, CS144

10

Noun Phrase Identification

• Identify the **nouns and noun phrases** in textual descriptions of a domain, and consider them as ***candidate*** conceptual classes or attributes.

• Use cases

Main Success Scenario (or Basic Flow):

1. **Customer** arrives at a **POS checkout** with **goods** and/or **services** to purchase.
 2. **Cashier** starts a new **sale**.
 3. **Cashier** enters **item identifier**.
 4. System records **sale line item** and presents **item description, price**, and running **total**. Price calculated from a set of price rules.
- Cashier repeats steps 2-3 until indicates done.
5. System presents total with **taxes** calculated.
 6. Cashier tells Customer the total, and asks for **payment**.
 7. Customer pays and System handles payment.
 8. System logs the completed **sale** and sends sale and payment information to the external **Accounting** (for accounting and **commissions**) and **Inventory** systems (to update inventory).
 9. System presents **receipt**.
 10. Customer leaves with receipt and goods (if any).

Extensions (or Alternative Flows):

...

Copyright © 2020 by Dr. Simon Xu,

11

Main Success Scenario (or Basic Flow):

1. **Customer** arrives at a **POS checkout** with **goods** and/or **services** to purchase.
 2. **Cashier** starts a new **sale**.
 3. **Cashier** enters **item identifier**.
 4. System records **sale line item** and presents **item description, price**, and running **total**. Price calculated from a set of price rules.
- Cashier repeats steps 2-3 until indicates done.
5. System presents total with **taxes** calculated.
 6. Cashier tells Customer the total, and asks for **payment**.
 7. Customer pays and System handles payment.
 8. System logs the completed **sale** and sends sale and payment information to the external **Accounting** (for accounting and **commissions**) and **Inventory** systems (to update inventory).
 9. System presents **receipt**.
 10. Customer leaves with receipt and goods (if any).

Extensions (or Alternative Flows):

...

Copyright © 2020 by Dr. Simon Xu,

12

Noun Phrase Identification

- Some of these noun phrases (NP) are **candidate conceptual classes**
- some may refer to conceptual classes that are ignored in this iteration (for example, "Accounting" and "commissions"), and

Copyright © 2020 by Dr. Simon Xu,

13

Noun Phrase Identification

- Different noun phrases may represent the same conceptual class or attribute, among other ambiguities (example)
 - **excluding abstract nouns** (may become class attributes) (examples)
 - **excluding nouns outside problem boundary**
- Some may be simply attributes of conceptual classes.

Copyright © 2020 by Dr. Simon Xu,

14

Example: Case Study: POS Domain

Sale	Cashier
CashPayment	Customer
SalesLineItem	Store
Item	ProductDescription
Register	ProductCatalog
Ledger	

Copyright © 2020 by Dr. Simon Xu,

15

Example: Case Study: POS Domain

- In practice, one can immediately draw a UML class diagram of the conceptual classes as we uncover them.**



Copyright © 2020 by Dr. Simon Xu,

16

Guideline: Report Objects - Include 'Receipt' in the Model?

•Should we include "Receipt" in domain model?



Copyright © 2020 by Dr. Simon Xu, GEM

17

Guideline: Report Objects - Include 'Receipt' in the Model?

•Should we include "Receipt" in domain model?

- No. In general, showing a **report of other information** in a domain model is not useful since all its information is derived or duplicated from other sources.
- Yes. On the other hand, **it has a special role in terms of the business rules: why?**



Copyright © 2020 by Dr. Simon Xu, GEM

18

Guideline: Report Objects - Include 'Receipt' in the Model?

- The decision:
 - Since **item returns** are not being considered in this iteration, **Receipt** will be excluded.



Copyright © 2020 by Dr. Simon Xu,

19

Guideline: A Common Mistake with Attributes vs. Classes

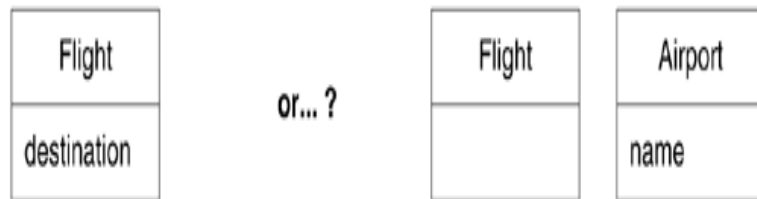
- The common mistake -- to represent something **as an attribute** when it should have been a **conceptual class**.
- A rule of thumb is:
 - *If we do not think of some conceptual class X as a **number** or **text** in the real world, X is probably a conceptual class, not an attribute*

Copyright © 2020 by Dr. Simon Xu,

20

Guideline: A Common Mistake with Attributes vs. Classes

- As an example, Should "destination" be an attribute of "Flight", or a separate conceptual class "Airport"?

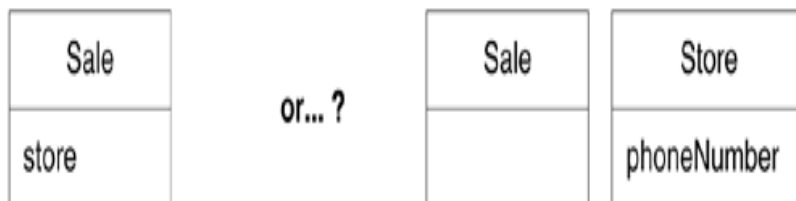


Copyright © 2020 by Dr. Simon Xu,
CC BY-SA 4.0

21

Guideline: A Common Mistake with Attributes vs. Classes

- As an example, should "store" be an attribute of "Sale", or a separate conceptual class "Store"?



Copyright © 2020 by Dr. Simon Xu,
CC BY-SA 4.0

22

Guideline: When to Model with 'Description' Classes?

- A **description** class contains information that describes something else.
 - For example, a *ProductDescription* that records the price, picture, and text description of an Item.
- **Why?**



Copyright © 2020 by Dr. Simon Xu, pptst.com

23

Guideline: When to Model with 'Description' Classes?

- Assume the following:
 - An Item instance represents a physical item in a store; as such, it may even have a serial number.
 - An Item has a **description, price, and itemID**, which are not recorded anywhere else.
 - Every time a real physical item is sold, a corresponding software instance of Item is deleted from "software land."

Copyright © 2020 by Dr. Simon Xu, pptst.com

24

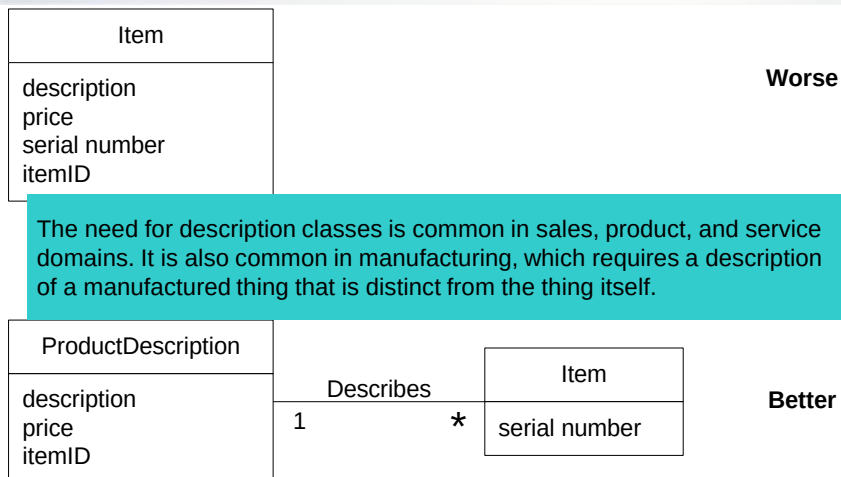
Guideline: When to Model with 'Description' Classes?

- To solve the Item problem, what is needed is a ***ProductDescription*** class that records information about items.
- A ProductDescription does not represent an Item, it represents **a description of information about items.**

Copyright © 2020 by Dr. Simon Xu,

25

Guideline: When to Model with 'Description' Classes?



Copyright © 2020 by Dr. Simon Xu,

26

Guideline: When are Description Classes useful?

- **Add a description class (for example, ProductDescription) when:**
 - There needs to be a description about an item or service, independent of the current existence of any examples of those items or services.
 - Deleting instances of things they describe (for example, Item) results in a loss of information that needs to be maintained, but was incorrectly associated with the deleted thing.
 - It reduces **redundant or duplicated** information.

Copyright © 2020 by Dr. Simon Xu,

27

Example: Descriptions in the Airline Domain

- **Consider an airline company that suffers a fatal crash of one of its planes.**
- **Assume that all the flights are cancelled for six months pending completion of an investigation.**
- **Also assume that when flights are cancelled, their corresponding Flight software objects are deleted from computer memory.**
- **What would happen?**

Copyright © 2020 by Dr. Simon Xu,

28

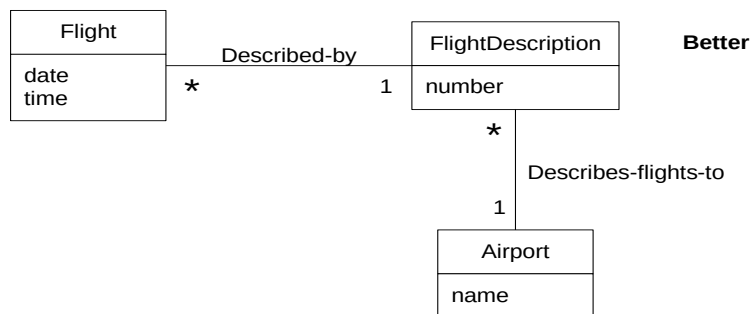
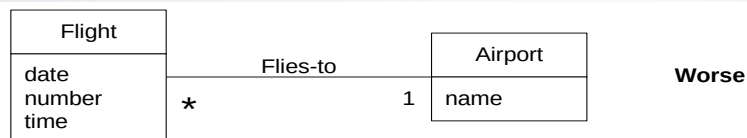
Example: Descriptions in the Airline Domain

- If the only record of what airport a flight goes to is in the **Flight software instances**, which represent specific flights for a particular date and time, then *there is no longer a record of what flight routes the airline has.*

Copyright © 2020 by Dr. Simon Xu,

29

Example: Descriptions in the Airline Domain

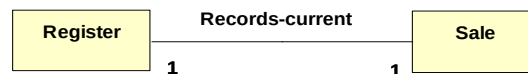


Copyright © 2020 by Dr. Simon Xu,

30

Step 2: Associations

- An association is a relationship between classes that indicates some meaningful and interesting connection.

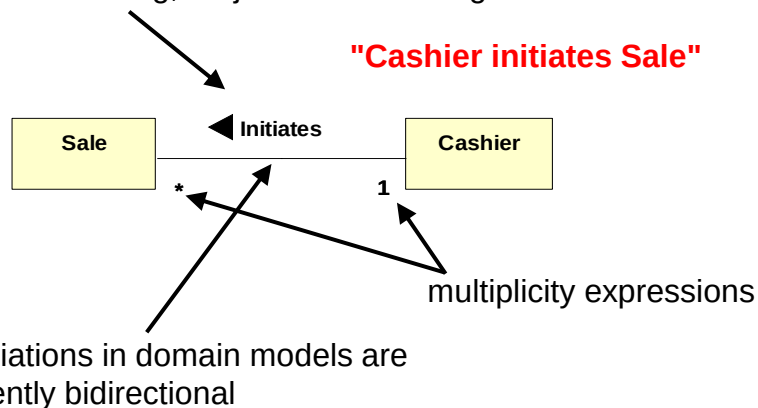


Copyright © 2020 by Dr. Simon Xu,

31

Associations

reading direction arrow
- no special meaning, it's just to aid reading



Copyright © 2020 by Dr. Simon Xu,

32

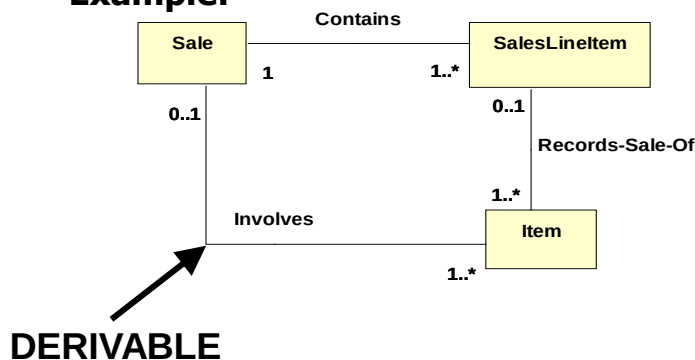
Finding Associations

- Consider including the following associations in a domain model:
 - Associations for which knowledge of the relationship needs to be preserved for some duration ("need-to-know" associations).
 - Associations derived from the [Common Associations List](#).
 - Give priority to these categories
 - A is a physical or logical part of B
 - A is physically or logically contained in/on B
 - A is recorded in B

Copyright © 2020 by Dr. Simon Xu, GMA

33

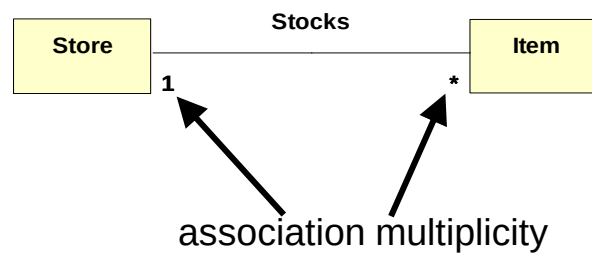
• Example:



Copyright © 2020 by Dr. Simon Xu, GMA

34

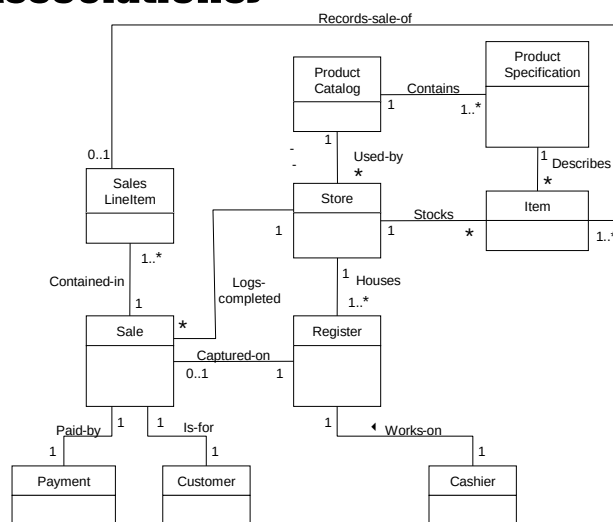
Step 3: Multiplicity



Copyright © 2020 by Dr. Simon Xu,

35

Domain Model For POS System (With Associations)



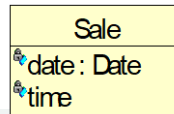
Copyright © 2020 by Dr. Simon Xu,

36

Step 4: Adding Attributes

- An attribute is a logical data value of an object.
- A domain model should include attributes for which the requirements (such as use cases) suggest or imply a need to remember information.

For example, a receipt normally includes the date and time of sale.



Copyright © 2020 by Dr. Simon Xu, CSU

37

- **Guidelines for Identifying Attributes**
The attributes in a domain model should be simple attributes or data types.

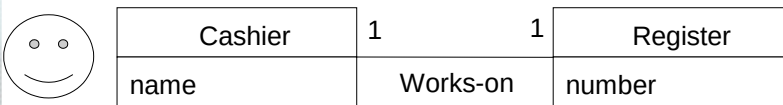
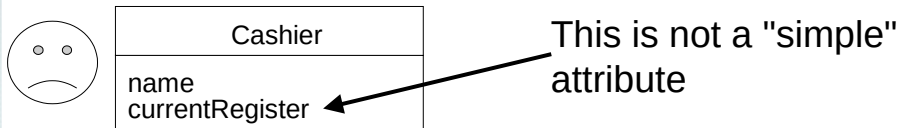
Examples of attribute types:

- Boolean, Date, Number, String (Text), Time
- Colour, PhoneNumber, SocialSecurityNumber, UniversalProductCode, Postcode

Copyright © 2020 by Dr. Simon Xu, CSU

38

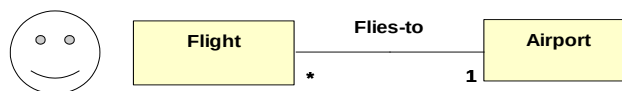
- Consider the following:



Copyright © 2020 by Dr. Simon Xu,

39

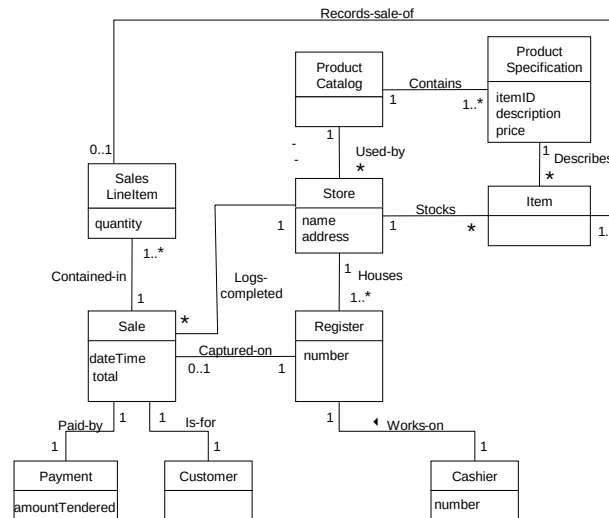
- Relate conceptual classes with an association, not with an attribute.



Copyright © 2020 by Dr. Simon Xu,

40

Domain Model for POS System



Copyright © 2020 by Dr. Simon Xu,

41

A Short Review Break

- **Domain model** To help understanding and more...
 - To understand the key concepts in a domain
 - To support a lower gap between the software representation and our mental model of the domain.
 - A key idea in OO.
- **Bounded by the current iteration requirements under design:**
 - Find the conceptual classes and Draw them as classes in a UML class diagram.
 - Add associations
 - Add multiplicities
 - Add attributes.
- **Guideline: Report Objects - Include 'Receipt' in the Model?**
- **Guideline: A Common Mistake with Attributes vs. Classes**
- **Guideline: When to Model with 'Description' Classes?**

Copyright © 2020 by Dr. Simon Xu.

42